

GSoC 2026 Proposal: Improve the new git repo command

Author: K Jayatheerth <jayatheerthkulkarni2005@gmail.com>

1. About Me

I am a junior Computer Science undergraduate at Geethanjali College of Engineering and Technology with a focus on systems programming.

My interest in the Git project stems from a desire to understand the internals of version control and contribute to a tool that is fundamental to the global software development ecosystem.

I am applying to this project specifically because I have followed the development of `builtin/repo.c` since its beginning in GSoC 2025. I have closely tracked its evolution through Lucas Oshiro's development blogs (<https://lucasoshiro.github.io/gsoc-en/>) and have studied the mailing list discussions.

2. General and Contact Information

- **Email:** jayatheerthkulkarni2005@gmail.com
- **GitHub:** github.com/jayatheerthkulkarni
- **Website:** jayatheerth.com
- **LinkedIn:** linkedin.com/in/jayatheerth
- **Timezone:** UTC+05:30 (IST)
- **Tech Stack:** C, Shell, Rust, Go

3. Previous Experience with Git

I have been an active contributor to the Git mailing list since February 2025. I have navigated the patch lifecycle, from submission and review to integration into `seen`, `next` and `master`.

My work has focused on areas directly relevant to this proposal.

I have been careful to avoid any microproject that would conflict with other GSoC applicants.

3.1 Selected Contributions

(1) [GSoC] t7101: modernize test path checks (Microproject)

Status: Merged into master

Link: [LINK](#)

Impact: Modernized legacy `test -f` checks to `test_path_is_file`. This is my official microproject for GSoC 2026.

(2) [PATCH v4]

- ` [PATCH v4 1/2] remote-curl: fall back to default hash outside repo
- ` [PATCH v4 2/2] refspect: fix typo in comment

Status: Merged into next

Link: [LINK](#)

Impact: Fixed a segmentation fault caused by an uninitialized hashing algorithm in the `ls-remote` command. Iterated the design based on active feedback from core maintainers to ensure architectural safety.

(3) [PATCH v3] dir.c: literal match with wildcard in pathspec should still glob

Status: Merged into master

Link: [LINK](#)

Impact: Fixed a logic error in `dir.c` where wildcard pattern matching terminated prematurely.

(4) [PATCH v3] stash: fix incorrect branch name in stash message

Status: Merged into master (Featured in Git Rev News #124)

Link: [LINK](#)

Link: [LINK](#)

Impact: Fixed a memory safety issue (static buffer reuse) in `refs_resolve_ref_unsafe` that corrupted stash messages in submodule environments.

(5) [PATCH v3 0/3] clean up a few things

- ` [PATCH v3 1/3] path: remove unused header
- ` [PATCH v3 2/3] path: use `size_t` for `dir_prefix` length
- ` [PATCH v3 3/3] path: remove redundant function calls

Status: Merged into master

Link: [LINK](#)

Impact: Improved readability and type safety in `path.c`.

- (6) [PATCH v8 0/2] Avoid submodule overwritten and skip redundant active entries
 ` [PATCH v8 1/2] submodule: prevent overwriting .gitmodules entry on path reuse
 ` [PATCH v8 2/2] submodule: skip redundant active entries when pattern covers path

Status: Merged into master

Link: [LINK](#)

Link (v9 and v10 rerolled in new thread): [LINK](#)

Impact: Prevented accidental overwrites of .gitmodules configuration when paths are reused.

- (7) [PATCH] repo: Remove unnecessary variable shadow

Status: Merged into master

Link: [LINK](#)

Impact: Executed a targeted cleanup in builtin/repo.c.

- (8) [PATCH v4 0/3] Update MyFirstContribution.adoc to follow modern practices
 ` [PATCH v4 1/3] docs: remove unused mentoring mailing list reference
 ` [PATCH v4 2/3] docs: clarify cmd _push signature and explain UNUSED macro
 ` [PATCH v4 3/3] docs: replace git _config to repo _config

Status: Merged into master

Link: [LINK](#)

Impact: Modernized documentation to reflect current API usage (e.g., repo_config).

3.2 Community Engagement & Design Discussions

Having navigated the mailing list learning curve, I actively participate in technical reviews to support the broader community.

- (1) Architecture Design: 'repo info' path normalization

Link: [LINK](#)

Context: Participated in discussions regarding path normalization strategies for the repo command.

- (2) Community Support: Patch Review & Onboarding

Selected Threads:

- [Thread 1](#)
- [Thread 2](#)
- [Thread 3](#)
- [Thread 4](#)
- [Thread 5](#)
- [Thread 6](#)

(3) Bug Investigation: 'git pull --rebase' fork-point behavior

Link: [LINK](#)

Context: Investigated the fork-point heuristic and identified core internal functions (`get_rebase_fork_point()` and `get_rebase_newbase_and_upstream()`) to explore potential logic fixes.

(4) Wildcards and Pathspec discussion

Link: [LINK](#)

Context: Investigated a bug where pathspecs were not expanding correctly. The discussion focused on `dir.c`, and the corresponding fix in my patch series originated from this thread.

4. Project Proposal

This project takes on an idea from the GSoC 2026 Ideas list. The main goal is to improve the new `git repo` command.

There are two main deliverables:

- a. Implement `path.*` keys for the `repo info` command.
- b. Implement category-based prefix querying for `repo info` keys.

Git takes code review seriously. Stability is the top priority for new features. To fit into this process and make sure the main goals get merged, I made the third objective a stretch goal and a post-GSoC commitment:

- c. Enhancing `repo structure` with Diagnostics (Stretch Goal).

Goal a: Path Resolution (Implement `path.*` keys for `repo info`)

The `path.*` keys are an active topic on the mailing list. The current effort is split. Lucas Oshiro started with the groundwork for path formatting. Eslam Reda has an overlapping patch series using a global `-path-format` flag.

However, the core architectural blocker remains unresolved. During these discussions ([LINK](#)), Brian M. Carlson established that: `git repo info` must return both an absolute and a relative path in the exact same call.

I participated in the discussion, stating that the two leading stateless solutions (Format Modifiers like `%(path:absolute)` and Computed Virtual Keys like `path.absolute.git-dir`) are basically the same approach.

My first priority during the Community Bonding period is to publish an RFC to reach consensus on the API syntax, decide default path values, and determine the best approach to compute both absolute and relative paths in a single invocation.

Once we agree on the API syntax, I will add support for these core paths:

- **path.git-dir:** The core repository directory (usually `.git`). Matches `rev-parse --git-dir`.
- **path.common-dir:** The base directory for shared data. Matches `rev-parse --git-common-dir`.
- **path.toplevel:** The absolute top-level directory of the working tree. Matches `rev-parse --show-toplevel`.
- **path.superproject-working-tree:** The root of the superproject's working tree if the current repository is a submodule. Matches `rev-parse --show-superproject-working-tree`.
- **path.objects:** The object database path. Accounts for the `GIT_OBJECT_DIRECTORY` variable.
- **path.hooks:** The hooks directory path. Respects `core.hooksPath` if set.
- **path.index:** The current index file path. Reflects `GIT_INDEX_FILE` overrides.
- **path.grafts:** The repository grafts file path.
- **path.git-prefix:** The path from the top-level of the working tree to the current directory. Matches `rev-parse --show-prefix`.

Risks and Patch Submission Style:

It might seem safe to submit one key, wait for it to merge into `next` or `master`, and then submit the next. But for a feature this big, that approach is too slow.

Git uses an asynchronous mailing list. A one-by-one strategy creates a fragile dependency chain. For example, `path.common-dir` could end up waiting weeks for `path.git-dir` to stabilize. More importantly, scattered patches stop reviewers from seeing the big picture. They need to see how the core parsing logic handles different edge cases all at once.

To respect the maintainers' time, I will submit the work as a single, organized patch series. It will be structured like this:

- **[PATCH 1/n]:** Introduces the core architecture change from the RFC. It includes the first key (`path.git-dir`). This shows real usage right away and avoids adding dead code.
- **[PATCH 2/n to n/n]:** Adds the remaining keys one by one. Each patch will only contain its specific code and tests. They will all build cleanly on top of Patch 1.

(Note: The series length might adjust if the architecture change is applied to `rev-parse` first, as Lucas suggested. This is just a tentative plan.)

Goal b: Category-Based Querying (Structured Discovery)

The `git repo info` command is growing. With the new `path.*` keys, the current "all-or-nothing" query model is a bottleneck. Users either need to know the exact key name, or they have to dump everything using `all` flag. While they can use tools like `grep` to filter out the results, given that `repo info` is a plumbing command, it is sloppy.

I will build a fast, prefix-based query system.

The internal structure currently looks like this:

```
static const struct repo_info_field repo_info_fields[] = {
    { "layout.bare", get_layout_bare },
    { "layout.shallow", get_layout_shallow },
    ...
};
```

The `repo_info_fields` array is static and sorted lexicographically. This means I can build a fast lookup mechanism without using heavy data structures like hash maps.

To handle this, I will write a custom $O(\log N)$ binary search. My implementation will locate the exact start of the prefix group, enforce strict namespace boundaries (matching `path.` but ignoring pathological), and yield the relevant keys.

After adding all paths, the array will hold roughly 13 elements. At this size, we are comparing an $O(N)$ linear scan to an $O(\log N)$ search. That is a maximum of 13 iterations versus just 4. A simple linear search is fast enough for that size, and the raw time difference is negligible right now.

However, this command is designed to grow. As we add future keys, the list will expand. Establishing a scalable foundation today prevents bottlenecks tomorrow.

Example: Querying the "layout" category

```
$ git repo info layout
layout.bare=false
layout.shallow=false
```

Design Rationale: Prefix Matching vs. Full Globbing

In early discussions, Karthik Nayak suggested supporting full glob patterns (like `git repo info *object*`). Globbing is flexible, but I am scoping this proposal to prefix matching for a simple reason: scale.

The `repo_info_fields` array currently has fewer than 5 elements. Even with all the new `path.*` keys, it will stay under 30.

Building a globbing engine is overkill for a command this small. However, during the Community

Bonding period, I will publish a short RFC. This will check if the community expects a massive influx of keys that might justify globbing later.

Risks and Patch Submission Style:

The main risk is namespace collision (accidentally matching the wrong substrings) or off-by-one errors in the binary search. To prevent this, I will rely on Git's existing string utilities and write strict tests for edge-case key names.

My submission strategy will isolate the algorithmic improvement from the user-facing API change. I will submit a targeted [PATCH 0/2] series:

- **[PATCH 1/2]:** Introduces the $O(\log N)$ lower-bound binary search helper function for the `repo_info_fields` array, with isolated unit tests to prove namespace boundary safety.
- **[PATCH 2/2]:** Wires the new search architecture into `cmd_repo_info` to enable category querying, updating documentation and `t1900` tests.

5. Timeline

I will dedicate 35-40 hours per week to this project. This is being counted as a "Large" sized project.

Community Bonding Period (May 1 - May 24, 2026)

- Publish my introductory GSoC blog post on jayatheerth.com/blogs.
- Publish the primary RFC to finalize the `path.*` API syntax (Format Modifiers vs. Computed Virtual Keys) to unblock Goal a.
- Publish the minor RFC regarding Prefix Matching vs. Full Globbing to establish community expectations for Goal b.
- Synchronize with Lucas Oshiro and mentors to ensure my plans align with any immediate upstream shifts.

Phase 1: Goal a Execution (May 25 – July 10, 2026)

This phase focuses strictly on Goal a (Path Resolution).

- **Weeks 1 - 4 (May 25 – June 21):**
 - Implement the core architectural foundation based on the RFC consensus (e.g., the `normalize_key` parser).
 - Implement all core `path.*` keys (`git-dir`, `common-dir`, `toplevel`, `superproject-working-tree`, `objects`, `hooks`, `index`, `grafts`, `git-prefix`).
 - Write comprehensive, OS-agnostic tests in `t/t1900-repo.sh` and update official documentation.
 - Milestone: Submit the complete, unified patch series (e.g., [PATCH 0/n]) encompassing the core architecture, all keys, and tests to begin the review cycle.

- **Weeks 5 - 7 (June 22 – July 10) [Buffer & Review Weeks]:**

- Dedicated buffer time for asynchronous review cycles. Address mailing list feedback for the Goal a patch series and submit subsequent versions ([PATCH v2], etc.).
- Midterm Evaluation: Aim to have the Goal a patch series stabilized and queued in next.
- Publish a detailed halfway-point blog post.

Phase 2: Goal b Execution (July 11 – August 21, 2026)

With Goal a stabilizing, development shifts entirely to Goal b (Category-Based Querying).

- **Weeks 8 - 10 (July 11 – July 31):**

- Implement the $O(\log N)$ lower-bound binary search helper function for the `repo_info_fields` array and write isolated unit tests to ensure strict namespace boundary enforcement.
- Wire the new search architecture into `cmd_repo_info` to enable category queries via the CLI.
- Expand `t1900` tests to ensure querying handles edge cases cleanly and update relevant documentation.
- Milestone: Submit the complete, unified patch series encompassing both the algorithmic foundation and the CLI integration.

- **Weeks 11 - 13 (August 1 – August 21) [Buffer & Review Weeks]:**

- Dedicated time for addressing potentially complex architectural reviews for the querying patch series.
- Submit subsequent patch versions ([PATCH v2], [PATCH v3]).
- Note: If reviews resolve early, I will gladly pivot to executing my stretch goal (Goal c: Enhancing repo structure with Diagnostics).

Phase 3: Final Handoff (August 22 – August 31, 2026)

- Write the final project report and publish the concluding blog post.
- Submit the Final GSoC Evaluation.

6. Availability

This timeline aligns perfectly with my schedule. The project kicks off in May and June, during which I will be on summer vacation and can dedicate 35-50 hours a week. During July, I will transition into my final year of university. My academic schedule during this period is highly flexible.

7. Blogging

I have a domain setup at jayatheerth.com. As patches flow and the project progresses, I will host a dedicated endpoint at /blogs to provide comprehensive, weekly, and phase-wise coverage of my project.

8. Stretch Goals & Post-GSoC Commitment

Goal c: Enhancing repo structure with Diagnostics (Stretch Goal)

Note: I will strictly not work on this stretch goal until I have fully completed and integrated Goal a and Goal b.

Initially, my plan for this stretch goal was to focus on libification by removing the global macro `USE_THE_REPOSITORY_VARIABLE` from `builtin/repo.c`. However, as of March 30, 2026, Patrick Steinhardt submitted an 18-part patch series ("[setup: drop uses of the_repository](#)") to drop uses of `the_repository` in `setup.c`. Since Patrick is actively dismantling the global repository state and mapping out the removal of `USE_THE_REPOSITORY_VARIABLE`, this libification effort will likely be completed prior to GSoC.

To ensure relevance, I am pivoting to a new feature. While `git repo structure` recently gained the ability to surface basic object counts and maximum sizes through Justin Tobler's recent patch series, the command still presents data as raw, isolated extremes.

My goal is to evolve `git repo structure` into a comprehensive diagnostic tool by introducing metric distributions:

- **Object Size and Entry Distributions (Histograms):**

As discussed recently on the mailing list by Patrick Steinhardt and Junio C Hamano, extreme maximums are useful, but distributions provide the real picture of repository health. I will implement a streaming bucketing system during the object walk to track size distributions (e.g., blob sizes) and entry distributions (e.g., tree entry counts). These will be formatted into an optional ASCII bar chart output to visualize repository shape.

Post-GSoC Involvement

I actively follow the mailing list and intend to continue contributing bug fixes and enhancements well beyond the summer. If the diagnostics work outlined in Goal c is not completed during the GSoC timeline, I would love to pick it up post-GSoC (provided the community agrees with the direction).

Having been a part of the Git community since early 2025, I plan to remain a long-term contributor. Eventually, I hope to give back to the community by mentoring future GSoC applicants if the opportunity arises.